Time Complexity: Step-by-Step Solutions

Problem 1

```
Code:
for (int i = 1; i <= n; i = i * 5) {
// computation
}
```

Solution (step-by-step):
1. Values produced by i: 1, 5, 5^2, 5^3, ..., 5^k where 5^k ≤ n < 5^(k+1).
2. The loop stops after k+1 iterations where k = floor(log_5 n).
3. So number of iterations = ■log_5 n■ + 1 = Θ(log_5 n).
4. By change-of-base formula, log_5 n = (log_2 n) / (log_2 5) = c · log_2 n where c is constant.
5. Therefore T(n) = Θ(log n).

Final: T(n) = Θ(log n) (equivalently O(log n), Ω(log n)).

Problem 2

```
Code:
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j = j * 2) {
// computation
}
}
```

Solution:
1. Outer loop (i) runs for i = 1..n → n iterations → Θ(n).
2. Inner loop (j) values: 1, 2, 4, ..., up to ≤ n → iterations ≈ ■log_2 n■ + 1 → Θ(log n).
3. Work per outer iteration = Θ(log n). Total = Θ(n) * Θ(log n) = Θ(n log n).

Final: T(n) = Θ(n log n).

Problem 3

```
Code:
for (int i = 1; i <= n; i = i * 2) {
for (int j = 1; j <= n; j = j * 3) {
// computation
}
}
```

Solution:
1. Outer loop i: values 1,2,4,... up to ≤ n → iterations ≈ ■log_2 n■ + 1 → Θ(log_2 n).
2. Inner loop j: values 1,3,9,... up to ≤ n → iterations ≈ ■log_3 n■ + 1 → Θ(log_3 n).
3. Total iterations = Θ(log_2 n) * Θ(log_3 n).
4. Using change-of-base both logs are Θ(log n), so product = Θ((log n)^2).

Final: T(n) = Θ((log n)^2).

Problem 4

```
Code:
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j++) {
for (int k = 1; k <= n; k = k * 2) {
// computation
}
```

```
}
}
```

Solution:
1. Outer loop i: n iterations → Θ(n).
2. Middle loop j: n iterations → Θ(n).
3. Inner loop k: 1,2,4,... ≤ n → Θ(log n) iterations.
4. Multiply: Θ(n) * Θ(n) * Θ(log n) = Θ(n^2 log n).

Final: T(n) = Θ(n^2 log n).

Problem 5

```
Code:
int m = n;
while (m > 1) {
m = m / 2;
for (int i = 1; i <= n; i++) {
// computation
}
}
```

Solution:
1. The while loop halves m each time: m, m/2, m/4, ... until ≤ 1.
2. Number of iterations of while ≈ ■log_2 n■ + 1 = Θ(log n).
3. Inside each iteration a for-loop runs n times → Θ(n) per while iteration.
4. Total work = Θ(n) * Θ(log n) = Θ(n log n).

Final: T(n) = Θ(n log n).

Problem 6

```
Code:
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= i; j = j * 2) {
// computation
}
}
```

Solution:
1. Fix i. The inner loop runs for j = 1,2,4,... ≤ i. Number of inner iterations ≈ ■log_2 i■ + 1 = Θ(log i).
2. Total = sum_{i=1}^{n} Θ(log i) = Θ(sum_{i=1}^{n} log i).
3. Note: sum_{i=1}^{n} log i = log(n!) (using log product rule).
4. Use Stirling's approximation: log(n!) = n log n - n + Θ(log n) = Θ(n log n).
- Alternatively, integral bound: ∑_{i=1}^{n} log i ≈ ∫_1^n log x dx = [x log x - x]_1^n = n log n - n + 1.
5. Therefore total = Θ(n log n). (The +1 per inner loop contributes Θ(n) extra, but dominated by n log n.)

Final: T(n) = Θ(n log n).

Problem 7

```
Code:
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j = j * 2) {
for (int k = 1; k <= n; k = k * 2) {
// computation
}
}
}
```

Solution:
1. Outer loop i: n iterations $\rightarrow \Theta(n)$.
2. Middle loop j: 1,2,4,... $\leq n \rightarrow \Theta(\log n)$.
3. Inner loop k: 1,2,4,... $\leq n \rightarrow \Theta(\log n)$.
4. Multiply: $\Theta(n) * \Theta(\log n) * \Theta(\log n) = \Theta(n (\log n)^2)$.

Final: $T(n) = \Theta(n (\log n)^2)$.